

PrivaLens: Automated Web Privacy & Regulatory Compliance Scanner

Project Proposal for UCS503P
Submitted to: Jeelani Asif
Thapar Institute of Engineering and Technology

Naman Arora, CSED* Prabhrajwin Singh, CSED[†] Ishmanjot Singh, CSED[‡]

August 19, 2026

1 Higher-order goal

The project aims to safeguard digital privacy rights and automate the verification of regulatory compliance (such as EU GDPR and India’s DPDP Act 2023). While these privacy frameworks impose strict legal mandates, small businesses, open-source developers, and startups frequently struggle to verify compliance due to complex third-party scripts. The immediate engineering objective of **PrivaLens** is to translate compliance auditing into a measurable, automated software verification solution.

2 Time-to-value

- We will deliver meaningful increments quickly by prototyping the headless traffic interception engine and validating PII detection accuracy early.
- We will prioritize fast feedback over perfection by using weekly iterations backed by CI/CD automation to test the compliance scoring algorithm.
- Success is defined by producing a fully functional single-page scan containing network logs and static policy parsing within the first iteration.

3 Problem Statement

Modern web applications integrate numerous external dependencies, widgets, and analytical trackers, exposing them to compliance violations. Key bottlenecks include:

- **PII Exfiltration:** Form fields and cookies transmit personal identifiable information (names, emails, IPs) to third-party endpoints.
- **Policy vs. Reality Mismatch:** Privacy policies verbally claim “we do not track users,” yet dynamic runtime telemetry actively transfers tracking keys to advertising brokers.

*Roll No: 1024160029, Email: namanarora.be24thapar.edu

[†]Roll No: 1024160024, Email: psingh.be24thapar.edu

[‡]Roll No: 1024160016, Email: isingh.be24thapar.edu

- **High Legal Overhead:** Startups lack the financial resources to hire specialized privacy auditing teams to review site updates.
- **Linter Limitations:** Static code analyzers fail to catch dynamic runtime traffic, third-party script loads, or tag manager modifications.

4 Proposed Solution

4.1 Overview

Build a **web-based** Privacy & Regulatory Compliance Scanner with the following components:

- **Scan Submission Portal:** An interactive dashboard where users input target URLs to initiate scans.
- **Headless Traffic Interceptor:** A headless browser environment that crawls target pages and records network interactions.
- **NLP Policy Analyzer:** An NLP pipeline that parses site privacy text to extract statements regarding user data collection.
- **Compliance & Discrepancy Scorecard:** A detailed dashboard rendering a compliance audit log and corrective suggestions.

4.2 Core workflow

1. Developer submits their website URL to the PrivaLens dashboard.
2. The backend adds the URL to the asynchronous scan queue (Redis/Bull).
3. A Puppeteer worker executes a headless crawl, logging all outgoing request payloads, cookies, and external domains.
4. A Python microservice parses the target site's Privacy Policy page to identify data-sharing clauses using text classification.
5. PrivaLens cross-references the live network logs against the NLP-parsed policy claims to build a discrepancy matrix.
6. The frontend displays a compliance scorecard with severity ratings and downloadable PDF remediation reports.

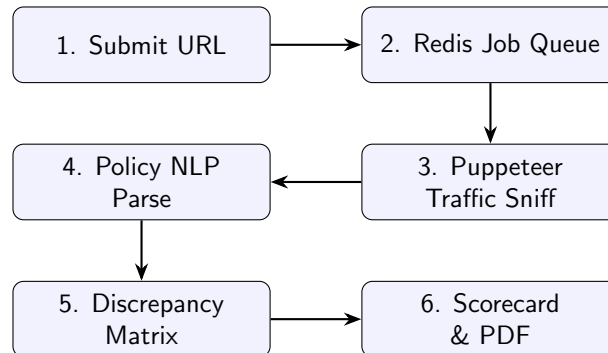


Figure 1: PrivaLens Core Workflow Execution Pipeline

4.3 Operational constraints for an educational setting

- Design the crawler to avoid overloading target servers; keep the scope limited to academic demonstration.
- Utilize lightweight NLP text classification libraries (such as SpaCy) to keep computation boundaries predictable.
- Ensure the application is deployable on containerized local staging or standard cloud virtual machines.

5 Solution Approach

This project focuses on building a resilient, asynchronous web system and legal-tech workflow, rather than novel machine learning architecture.

5.1 Discrepancy and compliance detection

Data mapping and mismatch flagging will be rule-based and classification-guided:

- Categorize network requests by target domain using community-maintained lists (e.g., EasyList, Disconnect).
- Use simple NLP text classifiers to identify policy paragraphs addressing collection categories (e.g., “analytics”, “marketing”).
- Compare the two sets: if marketing domains are active but the policy denies advertising cookies, flag a discrepancy.

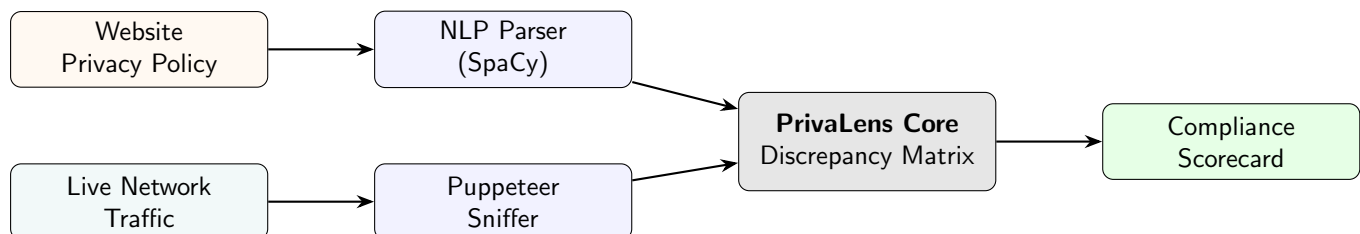


Figure 2: Discrepancy Matrix Analysis Logic

5.2 Web architecture

A decoupled 3-tier web architecture:

- **Frontend:** Next.js responsive client serving uploader forms, live progress bars, and scorecards.
- **Backend Worker Queue:** Node.js API server coordinating a Bull/Redis task queue for Puppeteer crawlers.
- **NLP Engine Storage:** Python FastAPI microservice handling policy text parsing, and a PostgreSQL database storing scan histories.

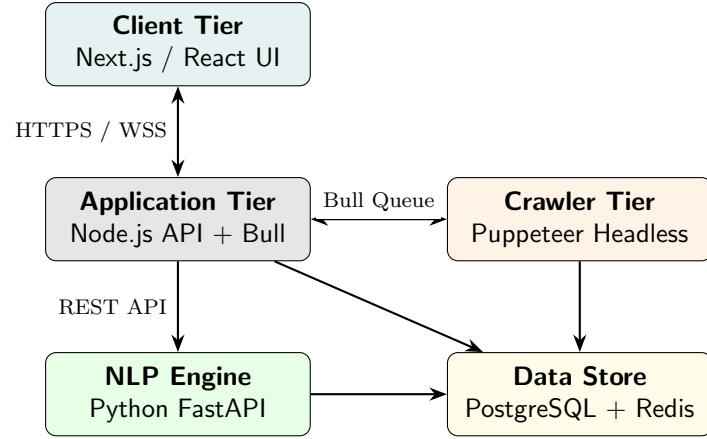


Figure 3: PrivaLens 3-Tier Microservice Web Architecture

5.3 CI/CD and fast delivery enabler

CI/CD pipelines will be configured to:

- Run automated unit tests on the network interceptor on every push.
- Build the Next.js static pages and verify endpoint health automatically.
- Publish updated MkDocs documentation to GitHub Pages via Git actions.

6 Evaluation Criterion

We define the success of PrivaLens using measurable project metrics.

6.1 Primary evaluation metric

Scan-to-Report Latency (SRL): The duration from when a user submits a URL to the generation of the compliance scorecard.

- Target: Median SRL ≤ 30 seconds for a 5-page crawl.
- Attribution: Measured via worker start and completion timestamps logged in PostgreSQL.

6.2 Secondary evaluation metrics

- **Request Capture Rate:** Percentage of outbound HTTP/WebSocket requests correctly intercepted and logged by Puppeteer compared to standard browser DevTools.
- **Policy Paragraph Recall:** Accuracy of the NLP classifier in identifying clauses related to data collection (Target $\geq 85\%$).
- **Job Completion Reliability:** Uptime and successful processing rate of the background job worker under concurrent loads.

6.3 Pilot validation plan

- Audit 10 popular web domains with known tracking configurations and published privacy policies.
- Verify compliance scanner accuracy by comparing PrivaLens scorecard alerts against manual network inspections.

7 Scalability

1. **Scalable (theoretically):** Decoupling the frontend from the resource-intensive crawler worker ensures UI requests remain responsive even under high scan loads.
2. **Operationally deployable (practically):** The complete system is containerized with Docker, enabling automated scaling of headless browser workers.

8 Engine availability heuristic

A mature set of software tooling is available for implementation:

- Puppeteer for browser automation and request interception.
- Bull and Redis for robust distributed task queues.
- SpaCy and transformers for policy classification.
- Next.js and Tailwind CSS for dashboard development.
- PostgreSQL for relational scan history storage.

9 Project scope and deliverables

9.1 Initial deliverable

- Headless crawler engine capturing HTTP traffic.
- Simple UI for scan initiation and basic progress logs.
- SQLite/PostgreSQL persistence for basic crawl summaries.
- Basic layout-checking parser for policy text.

9.2 Subsequent deliverables

- Asynchronous job queue integration (Bull/Redis).
- Text-classification model mapping clauses to tracking categories.
- Comprehensive PDF compliance scorecard export.
- Multi-page crawling and deep request payload analysis.

10 Risks and mitigations

- **Crawler Blocking:** Target sites using anti-bot/WAF configurations might block the Puppeteer scraper. Mitigated by utilizing custom user-agent strings and delayed request pacing.
- **Complex Policy Formatting:** Some privacy policies use irregular HTML layout formatting. Mitigated by using unified text extraction libraries that isolate raw readable paragraphs.

11 Summary

This project proposes **PrivaLens**, a deployable web service to automate data privacy and regulatory compliance checks. It fits the UCS503P syllabus by emphasizing time-to-value, asynchronous architecture, measurable performance metrics (Scan-to-Report Latency), and strict software engineering guidelines over research-driven algorithms.